

Concept Note

AI Supported Monitoring Pipeline for German Pension Policy. This note explains the approach to the overall problem statement. To identify relevant developments in German pension policy at early stage, it is important to prioritise the official sources such as Bundestage/DIP, Bundesrat, BMAS. Following strategy to be used along with an assisted AI component. The system should monitor regulatory developments, political topics, stakeholder activities and public debate.

Define scope, sources and monitoring strategy.

The first step is to define what constitutes a relevant pension policy development and establish what topics to start working with, such as statutory pensions, retirement age, contribution rates, occupational pensions and private retirement provision. I would prioritise official sources and distinguish primary sources (directly from official sources) from secondary information.

Prioritise Bundestag/DIP, Bundesrat, BMAS and Bundesregierung. I would like to have a configuration file with search terms, topic categories and metadata from sources instead of hardcoding in pipeline

Data collection from official sources

As data collection via API is most reliable form of data collection, first priority is to find out relevant APIs from the official sources for example DIP API key, APIs also gives data in structured format which is easy to process. Scraping will be used as a fallback where no suitable structured data collection is possible. Targeted HTML scraping/parsing works with website are updated and not changed frequently. Collecting data via online PDF wherever possible will be the next step in the process and last would be manually downloading for important and missing information to fill the gaps. I would consider authentication, rate limits, pagination, incremental retrieval and source availability

Tools: Python with requests or httpx for APIs and feedparser for RSS/XML. Use BeautifulSoup or lxml for targeted scraping and Playwright only where JavaScript rendering is required.

Download the raw data and store it

Raw data to be preserved in its original format so that it can be used for later purpose without requiring to fetch whole data or use API for each iteration. Tools like SQLite can be used for small database or prototype testing, raw data can also be stored in JSON format. However for large databases PostgreSQL to be used.

Validate, clean and transform

Different sources will have different structures and metadata quality so I would validate schemas, data types, required fields, duplicates and timestamps before combining sources.

Python and SQL/dbt will be used to transform the data and could follow raw – staging-- intermediate -- marts, with tests for uniqueness, missing values, relationships etc.

Orchestrate and monitor the pipeline

A monitoring system must run repeatedly and reliably. A failed API request must not be interpreted as no new political development. I would therefore consider scheduling, dependencies, retries, logging, data

freshness and failure alerts. I would use Dagster to orchestrate ingestion, validation, dbt transformations, AI enrichment and delivery, while providing pipeline observability

I would use deterministic methods wherever results need to be reproducible, explainable and auditable. These include document IDs, date and source filters, deduplication, controlled topic rules, legislative stage mapping and comparison of document versions. Deterministic prefiltering also reduces unnecessary AI processing. Tools: Python, SQL and dbt to be used for rule based filtering, identifiers, hashes, metadata comparison and change detection.

Use AI for semantic enrichment

AI should complement rather than replace the structured data layer. It is most valuable for tasks that are difficult to express reliably through fixed rules. AI to be used for determining substantive relevance, classifying topics, extracting stakeholders and policy positions, summarising documents and explaining substantive changes between versions.

Tools: Use an LLM/API with structured outputs and schema validation. AI generated information should be distinguishable from facts directly extracted from the source. Low confidence or high impact results should be routed to human review to validate and confirm

Creating early warnings

Not every new document deserves a client alert. I would combine source reliability, legislative status, deterministic relevance, detected changes and AI based semantic relevance to prioritise developments. Importantly, the system should distinguish between a change was detected and the change is potentially significant by applying high, medium and low priority thresholds and introduce human review for high impact developments.

Present results to clients

Different users may need different levels of detail, so I would provide layered outputs rather than a single report. For example, providing a real time alerts for high priority development via slack or mail alerts, daily/weekly briefing with key developments and finally as a long term solution a dashboard to show trends, legislative processes related to pensions.

Data collection, validation, deduplication, identifiers, timestamps, provenance, legislative metadata and basic change detection should remain deterministic because they need to be reproducible and auditable. AI should be used for semantic relevance, topic classification, entity and position extraction, summarisation and explaining substantive changes.

For production, Dagster would orchestrate the pipeline, **dbt** would manage reproducible transformations and data quality tests, and Docker could provide consistent runtime environments. Ansible could automate server configuration and deployment, while Git and GitHub would provide version control and documentation.